

Skill-06

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano escape.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc escape.c -o escape
himasree@LAPTOP-41COT5AA:~/OSLab$ ./escape
===== ESCAPE SEQUENCE TEST =====
Enter a string:
Hello\ World

Original input:
Hello\ World

===== ESCAPED CHARACTERS =====
New line: Hello\nWorld
Tab: Hello\tWorld
Backslash: C:\\Users\\Student
Double quote: \"Hello\"
Single quote: \'Hello\'

===== ESCAPED SPACE =====
Escaped space example: Hello\ World

===== SPECIAL SYMBOLS =====
Dollar: \$HOME
Ampersand: \&
Semicolon: \;
Pipe: \|

===== PARSER OUTPUT =====
Input received: Hello\ World
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano child_exec.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc child_exec.c -o child_exec
himasree@LAPTOP-41COT5AA:~/OSLab$ ./child_exec
===== CHILD PROCESS EXECUTION =====

Parent process.

Child process created.
Parent PID: 1235
Child PID: 1236
Child PID: 1236
Child will execute: ls -l
Parent waiting for child...

total 1448
-rwxr-xr-x 1 himasree himasree 16264 Aug 27 09:37 Apipes
-rw-r--r-- 1 himasree himasree  722 Aug 28 04:51 Apipes.c
-rw-r--r-- 1 himasree himasree  218 Aug 28 04:54 Apipes2.c
-rwxr-xr-x 1 himasree himasree 16048 Aug 28 04:53 Appipes2
-rwxr-xr-x 1 himasree himasree 16056 Aug 28 05:00 FIFO_CREATE
-rw-r--r-- 1 himasree himasree  220 Aug 28 04:59 FIFO_CREATE.c
-rwxr-xr-x 1 himasree himasree 16144 Aug 28 05:01 FIFO_READ
```

```
Child process finished.
Child exited normally.
Exit status: 0
```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ cat child_exec.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("==== CHILD PROCESS EXECUTION =====\n");

    // Create child process
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    // Child process
    if (pid == 0)
    {
        printf("\nChild process created.\n");

```

```

        printf("Child PID: %d\n", getpid());

        printf("Child will execute: ls -l\n\n");

        // Execute ls with argument -l
        char *args[] = {"ls", "-l", NULL};

        execvp(args[0], args);

        // This executes only if execvp fails
        perror("execvp failed");
        exit(1);
    }

    // Parent process
    else
    {
        printf("\nParent process.\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent waiting for child...\n");

        if (waitpid(pid, &status, 0) == -1)
        {
            perror("waitpid failed");
            return 1;
        }

        printf("\nChild process finished.\n");

```

```

    }
    perror("waitpid failed");
    return 1;
}

printf("\nChild process finished.\n");

if (WIFEXITED(status))
{
    printf("Child exited normally.\n");
    printf("Exit status: %d\n", WEXITSTATUS(status));
}
else
{
    printf("Child did not exit normally.\n");
}
}

return 0;

```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ cat child_exec.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("==== CHILD PROCESS EXECUTION =====\n");

    // Create child process
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    // Child process
    if (pid == 0)
    {
        printf("\nChild process created.\n");
        printf("Child PID: %d\n", getpid());

        printf("Child will execute: ls -l\n\n");
    }
}

```

```

    execvp(args[0], args);

    // This executes only if execvp fails
    perror("execvp failed");
    exit(1);
}

// Parent process
else
{
    printf("\nParent process.\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("Parent waiting for child...\n");

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        return 1;
    }

    printf("\nChild process finished.\n");

    if (WIFEXITED(status))
    {
        printf("Child exited normally.\n");
        printf("Exit status: %d\n", WEXITSTATUS(status));
    }
}

```

```

    {
        printf("Child exited normally.\n");
        printf("Exit status: %d\n", WEXITSTATUS(status));
    }
    else
    {
        printf("Child did not exit normally.\n");
    }
}

return 0;
}

```